

PENETRATION TEST REPORT

Web Application Security Assessment

Product Bulk Upload

1. Executive Summary

A security assessment was performed against [Client / Application Name], a Django-based web application, focusing on authentication, session management, access control, CSRF protection, SQL injection, cross-site scripting (XSS), and file upload handling. Testing was conducted using Django's test client against the application's request/response and ORM layers.

Of 16 test cases executed, 13 demonstrated secure, expected behavior, 3 returned inconclusive results due to HTTP 404/302 responses that require re-testing against confirmed live endpoints and authenticated sessions, and 0 confirmed exploitable vulnerabilities.

The most significant positive finding is that the application's Django ORM layer correctly parameterizes queries, preventing classic SQL injection even when payloads were passed directly to model filters (TC-09). CSRF protection, session management, and access control on the dashboard route were also confirmed to be functioning correctly.

The highest-priority open item is file upload validation (TC-16): the test could not confirm whether file type, content, and size restrictions are enforced, because the request appears to have been redirected due to a missing authenticated session rather than a validation control. This should be re-tested as a priority before this control can be considered verified.

Overall Risk Posture

ID	Test Case	Category	Result	Risk
TC-01	Baseline Login Attempt via HTTP POST	Authentication	Secure	Low
TC-02	Route / Response Code Baseline	Reconnaissance	Secure	Informational
TC-03	CSRF Enforcement on Safe (GET) Requests	CSRF Protection	Secure	Informational
TC-04	Authenticated Dashboard Access / Redirect Chain	Access Control	Secure	Informational
TC-05	Session Invalidation on Logout	Session Management	Secure	Low
TC-06	Valid Credential Authentication	Authentication	Secure	Informational
TC-07	SQL Injection — Login Form Field	Input Validation (SQLi)	Inconclusive	Informational
TC-08	Cross-Site Scripting — Upload Comment Field	Input Validation (XSS)	Inconclusive	Informational
TC-09	SQL Injection — Django ORM Layer	Input Validation (SQLi)	Secure	Low

ID	Test Case	Category	Result	Risk
TC-10	CSRF Protection on State-Changing POST	CSRF Protection	Secure	Low
TC-11	SQL Injection — Username & Password Fields (HTTP)	Input Validation (SQLi)	Secure	Low
TC-12	Unauthenticated Access to Protected Route	Access Control	Secure	Low
TC-13	CSRF Cookie Attribute Review	CSRF Protection	Secure	Medium
TC-14	Anonymous User State Prior to Login	Session Management	Secure	Informational
TC-15	Session Fixation / Auth Hash Verification	Session Management	Secure	Low
TC-16	Malicious Executable File Upload	File Upload Security	Inconclusive	High (pending re-test)

2. Detailed Test Results

TC-01 — Baseline Login Attempt via HTTP POST

Category	Authentication
Objective	Confirm behavior of the login endpoint when credentials are submitted directly over HTTP.
Method	Used Django's test Client to POST an email and password to the application root ('/').
Observed Result	Server responded with HTTP 200 (form re-rendered) rather than a redirect, with no verbose error or account-existence information disclosed in the response code.
Status	Secure
Risk Rating	Low
Analysis / Recommendation	No sensitive information (e.g., 'invalid password' vs 'user not found') was distinguishable from the status code alone. Recommend confirming the rendered page body also avoids user-enumeration hints.

Evidence:

```
Ctrl click to launch VS Code Native REPL
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from django.test import Client
>>> c=Client()
>>> res=c.post('/', {'email': 'sivaprakash', 'password': 'Ajith@123'})
...
... )
>>> res.status_code
200
>>> []
```

TC-02 — Route / Response Code Baseline

Category	Reconnaissance
Objective	Establish baseline HTTP response codes for valid and invalid routes.
Method	Issued GET requests to '/' and a deliberately invalid path ('login/incorrect').
Observed Result	'/' returned HTTP 200; the invalid path returned HTTP 404 as expected, with no stack trace or debug information leaked.
Status	Secure
Risk Rating	Informational
Analysis / Recommendation	Standard, expected routing behavior. No evidence of DEBUG=True or verbose error pages being exposed to unauthenticated users.

Evidence:

```
>>> c.get('/')
<HttpResponse status_code=200, "text/html; charset=utf-8">
>>> c.get('login/incorrect')
<HttpResponse status_code=404, "text/html; charset=utf-8">
>>>
```

TC-03 — CSRF Enforcement on Safe (GET) Requests

Category	CSRF Protection
Objective	Verify that enabling strict CSRF enforcement does not break normal navigation, and that a spoofed User-Agent has no bearing on access control.
Method	Created a Client with a spoofed 'curl/7.79.1' User-Agent header and a separate Client with enforce_csrf_checks=True, then issued GET requests.
Observed Result	Both requests returned HTTP 200. GET requests are correctly exempt from CSRF token requirements (per Django design), and no User-Agent based bypass was observed.
Status	Secure
Risk Rating	Informational
Analysis / Recommendation	Confirms CSRF middleware is active on the application. This test does not by itself prove state-changing (POST) requests are protected — see TC-10 for that evidence.

Evidence:

```
>>> client = Client(headers={"User-Agent": "curl/7.79.1"})
>>> csrf_client = Client(enforce_csrf_checks=True)
>>> response = csrf_client.get("/")
>>> print(response.status_code)
200
>>>
```

TC-04 — Authenticated Dashboard Access / Redirect Chain

Category	Access Control
Objective	Confirm that an authenticated session reaches the dashboard without unexpected intermediate redirects.
Method	Following a successful login, requested '/dashboard/' with follow=True and inspected the redirect chain.
Observed Result	The redirect chain was empty, indicating the dashboard was served directly (HTTP 200) once authenticated.

Status	Secure
Risk Rating	Informational
Analysis / Recommendation	Expected behavior for an authenticated session. No open-redirect or unexpected intermediate hop was present.

Evidence:

```
>>> res=c.get('dashboard/',follow=True)
>>> res.redirect_chain
[]
>>>
```

TC-05 — Session Invalidation on Logout

Category	Session Management
Objective	Verify that logging out revokes access to protected pages.
Method	Called client.logout() and then requested '/dashboard/'.
Observed Result	Server responded with HTTP 302, redirecting to '/?next=/dashboard/' — the session was correctly invalidated and access control enforced.
Status	Secure
Risk Rating	Low
Analysis / Recommendation	Correct behavior. The 'next' parameter should be validated server-side to ensure only local, relative redirects are honored (Django's default login flow does this), preventing potential open-redirect abuse.

Evidence:

```
>>> c.logout()
>>>
>>> res=c.get('/dashboard/')
>>> res.status_code
302
>>> res.url
'/?next=/dashboard/'
>>>
```

TC-06 — Valid Credential Authentication

Category	Authentication
----------	----------------

Objective	Baseline confirmation that a known-valid account can authenticate.
Method	Used client.login(username='sivaprakash', password='Siva@123').
Observed Result	Authentication returned True, confirming the credential set used for subsequent authenticated test cases.
Status	Secure
Risk Rating	Informational
Analysis / Recommendation	Baseline/setup step for later authenticated tests (TC-04, TC-15). No issue identified.

Evidence:

```

type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from django.test import Client
>>> c=Client()
>>> c.login(username='sivaprakash',password='Siva@123')
True
>>>

```

TC-07 — SQL Injection — Login Form Field

Category	Input Validation (SQLi)
Objective	Attempt classic SQL injection (' OR 1=1 --) against the login form's identifier field.
Method	POSTed a payload of "' OR 1=1 --" in the 'email_or_username' field with an arbitrary password to the root endpoint.
Observed Result	Server responded with HTTP 404.
Status	Inconclusive
Risk Rating	Informational
Analysis / Recommendation	A 404 response indicates the submitted field name or endpoint path did not match the application's actual login route, rather than confirming the payload was sanitized. Recommend re-running this test against the confirmed, correct login endpoint and field names before drawing a security conclusion. TC-09 provides stronger, definitive evidence that the ORM layer itself is not injectable.

Evidence:

```
>>> response = client.post("/login/", {
...     "email_or_username": "' OR 1=1 --",
...     "password": "anything",
... })
>>> print(response.status_code)
404
>>>
```

TC-08 — Cross-Site Scripting — Upload Comment Field

Category	Input Validation (XSS)
Objective	Attempt to store a script payload (<script>alert('XSS')</script>) via a file-upload comment field.
Method	POSTed the payload as a 'comment' field to '/upload/'.
Observed Result	Server responded with HTTP 404.
Status	Inconclusive
Risk Rating	Informational
Analysis / Recommendation	As with TC-07, a 404 suggests the endpoint or field name assumed for this test does not match the live route (e.g., authentication may be required first, or the path differs). This does not confirm the field is safe from stored XSS. Recommend re-testing against the correct, authenticated upload endpoint and verifying output encoding on any page that later renders the comment.

Evidence:

```
>>> payload = {
...     "comment": "<script>alert('XSS')</script>"
... }
>>> res=client.post('upload/',payload)
>>> print(res.status_code)
404
>>>
```

TC-09 — SQL Injection — Django ORM Layer

Category	Input Validation (SQLi)
Objective	Determine whether the application's ORM layer is vulnerable to SQL injection when a classic payload is used as a filter value.
Method	Directly queried the User model with username/email filters using the payload "' OR 1=1 --", and printed the resulting compiled SQL query.

Observed Result	Zero records were returned (users.count() == 0, users.exists() == False, User.DoesNotExist raised). The printed query confirms the payload was passed as a bound, escaped string literal in a parameterized WHERE clause, not interpreted as SQL syntax.
Status	Secure
Risk Rating	Low
Analysis / Recommendation	This is the strongest evidence in the assessment: Django's ORM parameterizes queries by default, and no injection was possible through this code path. Continue to avoid raw SQL / string-formatted queries (e.g., .raw() or extra()) elsewhere in the codebase to preserve this protection.

Evidence:

```
>>> print(response.url if hasattr(response, "url") else "No redirect")
No redirect
>>> from django.contrib.auth import get_user_model
>>> User = get_user_model()
>>> payload = "' OR 1=1 --"
>>> users = User.objects.filter(username=payload)
>>> print(users.count())
0
>>> print(list(users))
[]
>>> users = User.objects.filter(email=payload)
>>> print(users.exists())
False
>>> try:
...     user = User.objects.get(username=payload)
... except User.DoesNotExist:
...     print('no match')
...
No match
>>> qs = User.objects.filter(username=payload)
>>> print(qs.query)
SELECT "auth_user"."id", "auth_user"."password", "auth_user"."last_login", "auth_user"."is_superuser", "auth_user"."username", "auth_user"."first_name", "auth_user"."last_name", "auth_user"."email", "auth_user"."is_staff", "auth_user"."is_active", "auth_user"."date_joined" FROM "auth_user" WHERE "auth_user"."username" = ' OR 1=1 --'
>>>
```

TC-10 — CSRF Protection on State-Changing POST

Category	CSRF Protection
Objective	Confirm that a POST request lacking a valid CSRF token is rejected.
Method	Submitted a POST with an invalid username field and no CSRF token to the root endpoint.
Observed Result	Server responded with HTTP 403.
Status	Secure
Risk Rating	Low
Analysis / Recommendation	Correct behavior — Django's CSRF middleware is actively rejecting unprotected POST requests. No action required.

Evidence:

```
>>> res=client.post('/', {'username': 'invalid'})
>>> print(res.status_code)
403
```

TC-11 — SQL Injection — Username & Password Fields (HTTP)

Category	Input Validation (SQLi)
Objective	Attempt SQL injection via both username and password fields in a single POST.
Method	POSTed "' OR 1=1 --" as both username and password to the root endpoint.
Observed Result	Server responded with HTTP 403.
Status	Secure
Risk Rating	Low
Analysis / Recommendation	The request was rejected at the CSRF layer before reaching the query layer, and TC-09 independently confirms the underlying ORM query would not have been injectable regardless. No vulnerability identified.

Evidence:

```
>>> res= client.post("/", {
...     "username": "' OR 1=1 --",
...     "password": "' OR 1=1 --"
... })
>>> print(res.status_code)
403
```

TC-12 — Unauthenticated Access to Protected Route

Category	Access Control
Objective	Confirm that the dashboard cannot be accessed without an authenticated session.
Method	Issued a fresh GET request to '/dashboard/' with no prior authentication.
Observed Result	Server responded with HTTP 302 (redirect to login).
Status	Secure
Risk Rating	Low
Analysis / Recommendation	Correct access-control enforcement. No unauthenticated access to protected content was possible.

Evidence:

```
>>> response = client.get("/dashboard/")
>>> print(response.status_code)
302
```

TC-13 — CSRF Cookie Attribute Review

Category	CSRF Protection
Objective	Inspect the attributes of the CSRF token cookie for secure configuration.
Method	Requested '/' and inspected the Set-Cookie header for the csrftoken cookie.
Observed Result	Cookie included Path=/, SameSite=Lax, and a Max-Age of ~1 year (31,449,600 seconds).
Status	Secure
Risk Rating	Medium
Analysis / Recommendation	SameSite=Lax and a scoped Path are good defaults. The CSRF cookie is intentionally not HttpOnly by Django design (JavaScript must read it to set the X-CSRFToken header), which is expected — not a flaw. However, confirm the 'Secure' flag is set in the production environment (only enforced when the request is served over HTTPS with CSRF_COOKIE_SECURE=True); this could not be fully verified from a local test-client session. Recommend explicitly confirming CSRF_COOKIE_SECURE and SESSION_COOKIE_SECURE are enabled in production settings.

Evidence:

```
>>> response = client.get("/")
>>> print(response.cookies)
set-cookie: csrftoken=wJYzp7CP3Fx0EP5BxMek34MXBcQAvDLT; expires=Thu, 15 Jul 2027 09:23:19 GMT; Max-Age=31449600; Path=/; SameSite=Lax
>>>
```

TC-14 — Anonymous User State Prior to Login

Category	Session Management
Objective	Confirm the request user object correctly reflects an unauthenticated state before login.
Method	Requested '/' and '/dashboard/' with a fresh client and inspected wsgi_request.user.
Observed Result	The user object was correctly identified as AnonymousUser prior to authentication.
Status	Secure
Risk Rating	Informational
Analysis / Recommendation	Expected baseline behavior confirming the authentication middleware is functioning correctly.

Evidence:

```
>>> response = client.get("/")
>>> print(response.cookies)
Set-Cookie: csrftoken=WJYzp7CP3Fx0EP5BxMek34MXBcQAvDlT; expires=Thu, 15
Jul 2027 09:23:19 GMT; Max-Age=31449600; Path=/; SameSite=Lax
>>> response = client.get("/dashboard/")
>>> print(response.wsgi_request.user)
AnonymousUser
```

TC-15 — Session Fixation / Auth Hash Verification

Category	Session Management
Objective	Verify that a successful login establishes a session bound to a password-derived hash (protection against session fixation).
Method	Logged in via client.login() and inspected the resulting session dictionary.
Observed Result	Session contained <code>_auth_user_id</code> , <code>_auth_user_backend</code> , and <code>_auth_user_hash</code> — the latter is derived from the user's password and session secret.
Status	Secure
Risk Rating	Low
Analysis / Recommendation	Django's auth hash mechanism invalidates existing sessions if the password changes, which mitigates session fixation and stale-session risks. No issue identified.

Evidence:

```
>>> client.login(username='sivaprakash', password='Siva@123')
True
>>> print(dict(client.session))
{'_auth_user_id': '3', '_auth_user_backend': 'django.contrib.auth.backends.ModelBackend', '_auth_user_hash': '4a06b23346c339b735f9fba3d86572bb51901e79b040a710c1571c2775ef3ef'}
>>>
```

TC-16 — Malicious Executable File Upload

Category	File Upload Security
Objective	Attempt to upload a disguised executable (named 'virus.exe', declared as application/octet-stream) to the upload endpoint.
Method	POSTed a SimpleUploadedFile object named 'virus.exe' with fabricated content to '/upload/'.
Observed Result	Server responded with HTTP 302 (redirect).
Status	Inconclusive
Risk Rating	High (pending re-test)
Analysis / Recommendation	A 302 most likely indicates the request was redirected to a login page because this client session was not authenticated at the time of the request, rather than confirming file-type validation occurred. This is the highest-priority item to re-test:

	repeat the upload using an authenticated session and confirm the application enforces (a) file extension/content-type whitelisting, (b) content-based file type verification (not just declared MIME type), (c) storage outside any web-executable path, and (d) a file size limit. Until confirmed, treat file-upload validation as unverified.
--	--

Evidence:

```
>>> from django.core.files.uploadedfile import SimpleUploadedFile
>>> file = SimpleUploadedFile(
...     "virus.exe",
...     b"Fake EXE",
...     content_type="application/octet-stream"
... )
>>> response = c.post(
...     "/upload/",
...     {
...         "file": file
...     }
... )
>>> print(response.status_code)
302
>>>
```